Practical Set- 1

1. **Develop a Java program that prompts the user to enter a distance in meters. Your program should then convert this distance to feet (1 meter = 3.28084 feet) and display the result formatted to two decimal places.**

Code:
```
import java.util.Scanner;
 public class DistanceConverter {
 public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enter distance in meters: ");
double meters = sc.nextDouble();
double feet = meters * 3.28084;
System.out.printf("Distance in feet: %.2f\n", feet);
  }
}
```

2. **Write a Java program to solve a system of two linear equations with two variables (e.g., ax + by = e and cx + dy = f). Prompt the user to enter the coefficients a, b, c, d, e, f. Calculate and display the values of x and y using Cramer's rule. Include error handling for cases where the denominator is zero Cramer's rule : D=ad−bc, Dx=ed−bf, Dy=af−ce, x=Dx/D, y=Dy/D**

Code:
```
import java.util.Scanner;

public class CramerRule {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      double a = sc.nextDouble();
```

```
        double b = sc.nextDouble();
        double c = sc.nextDouble();
        double d = sc.nextDouble();
        double e = sc.nextDouble();
        double f = sc.nextDouble();

        double D = a*d - b*c;

        if (D == 0) {
            System.out.println("No unique solution");
        } else {
            double Dx = e*d - b*f;
            double Dy = a*f - e*c;

            double x = Dx / D;
            double y = Dy / D;

            System.out.println("x = " + x + ", y = " + y);
        }
    }
}
```

3. **Write a Java program that prompts the user to enter a single letter (character). Determine whether the entered character is a vowel (a, e, i, o, u, case-insensitive) or a consonant, and display the result.**

Code:
```
import java.util.Scanner;

public class VowelCheck {
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        char ch = sc.next().toLowerCase().charAt(0);

        if ("aeiou".indexOf(ch) != -1)
            System.out.println("Vowel");
        else
            System.out.println("Consonant");
    }
}
```

4. **Develop a Java application that calculates a person's Body Mass Index (BMI). The program should ask the user for their weight in pounds and height in inches. Convert these values to kilograms and meters respectively (1 pound = 0.45359237 kg, 1 inch = 0.0254 meters) and then calculate BMI (weight in kg / (height in meters)^2). Display the calculated BMI.**

Code:
```
import java.util.Scanner;

public class BMI {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        double pounds = sc.nextDouble();
        double inches = sc.nextDouble();

        double kg = pounds * 0.45359237;
        double meters = inches * 0.0254;
```

```
        double bmi = kg / (meters * meters);
        System.out.println("BMI: " + bmi);
    }
}
```

5. **Write a program that takes the lengths of three sides of a triangle as input. Calculate and print the area of the triangle. Ensure that the program validates if the given side lengths can actually form a triangle (sum of any two sides must be greater than the third side) before calculating the area (use Heron's formula: Area = sqrt(s*(s-a)*(s-b)*(s-c)) where s = (a+b+c)/2).**

Code:
```
import java.util.Scanner;

public class TriangleArea {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        double a = sc.nextDouble();
        double b = sc.nextDouble();
        double c = sc.nextDouble();

        if (a + b > c && a + c > b && b + c > a) {
            double s = (a + b + c) / 2;
            double area = Math.sqrt(s * (s-a) * (s-b) * (s-c));
            System.out.println("Area: " + area);
        } else {
            System.out.println("Invalid triangle");
        }
```

```
    }
}
```

Practical Set-2

6. **Define a Java class named Rectangle. It should have two double data fields: width and height, both with a default value of 1. Implement a no-argument constructor and a constructor that takes width and height as parameters. Include methods getArea() and getPerimeter() that return the calculated area and perimeter respectively.**

Code:
```
class Rectangle {
    double width = 1, height = 1;

    Rectangle() {}

    Rectangle(double w, double h) {
        width = w;
        height = h;
    }

    double getArea() {
        return width * height;
    }

    double getPerimeter() {
        return 2 * (width + height);
    }
}
```

7. **Create an Employee class with private instance variables for employeeName (String) and employeeSalary (double). Implement public methods readEmployeeData() (to take input from the user) and displayEmployeeData() (to print the employee's name and salary). Demonstrate object creation and method invocation in a main method**.

Code:
```
import java.util.Scanner;


class Employee {
    private String name;
    private double salary;

    void readEmployeeData() {
        Scanner sc = new Scanner(System.in);
        name = sc.nextLine();
        salary = sc.nextDouble();
    }

    void displayEmployeeData() {
        System.out.println(name + " " + salary);
    }

    public static void main(String[] args) {
        Employee e = new Employee();
        e.readEmployeeData();
        e.displayEmployeeData();
    }
}
```

8. **Create a Point class representing a 2D point (x, y). Implement a default constructor that initializes both x and y to 5. Provide a parameterized**

constructor to initialize x and y with user-supplied values. Also, implement a copy constructor to create a new Point object as a copy of an existing Point object. Include a display() method to show the point's coordinates and write a main method to test all constructors and the display functionality.

Code: class Point {

```
int x, y;

Point() {
    x = y = 5;
}

Point(int x, int y) {
    this.x = x;
    this.y = y;
}

Point(Point p) {
    x = p.x;
    y = p.y;
}

void display() {
    System.out.println(x + "," + y);
}

public static void main(String[] args) {
    Point p1 = new Point();
    Point p2 = new Point(2,3);
```

```
        Point p3 = new Point(p2);

        p1.display();
        p2.display();
        p3.display();
    }
}
```

9. **Define a Java class named Rectangle. It should have two double data fields: width and height. In your main method, create two Rectangle objects: one with width 4 and height 40, and another with width 3.5 and height 35.9. For each rectangle, display its width, height, calculated area, and perimeter. Then, compare the two rectangles based on their areas and print which one has a larger area.**

<u>Code</u>:
```
class RectangleTest {
    double width, height;

    RectangleTest(double w, double h) {
        width = w;
        height = h;
    }

    double area() { return width * height; }

    public static void main(String[] args) {
        RectangleTest r1 = new RectangleTest(4,40);
        RectangleTest r2 = new RectangleTest(3.5,35.9);

        System.out.println(r1.area());
        System.out.println(r2.area());
```

```
        if (r1.area() > r2.area())
            System.out.println("r1 larger");
        else
            System.out.println("r2 larger");
    }
}
```

Practical Set – 3

10. Design a class BankAccount with account_holder_name and balance. Use a static variable interest_rate (same for all accounts). Include methods to calculate and display the interest earned.SHREE SWAMI ATMANAND SARASWATI INSTITUTE OF TECHNOLOGY COMPUTER ENGINEERING DEPARTMENT Update interest rate using a static method.

Code:
```
class BankAccount {
    String name;
    double balance;
    static double interestRate = 5;

    BankAccount(String n, double b) {
        name = n;
        balance = b;
    }

    void calculateInterest() {
        double interest = (balance * interestRate) / 100;
        System.out.println("Interest: " + interest);
```

```
    }

    static void setRate(double r) {
        interestRate = r;
    }

    public static void main(String[] args) {
        BankAccount b1 = new BankAccount("A", 1000);
        b1.calculateInterest();
        setRate(10);
        b1.calculateInterest();
    }
}
```

11. Write a Java program to model a college admission system using the concept of inner classes.Create an outer class named College that stores the collegeName as a data member and initializes it through a constructor. Within the College class, define a non-static inner class named Admission that contains student-specific details such as the studentName and the course they are enrolling in.The inner class should have methods to accept and display student information, and it should also be able to access and display the collegeName stored in the outer class. In the main method, create an object of the College class by passing the collegeName, and then use this object to create an instance of the inner Admission class. Invoke appropriate methods to input the student's name and course, and then display the complete admission details, including the college name.

Code**:**

```
class College {
    String collegeName;

    College(String name) {
        collegeName = name;
```

```
    }

    class Admission {
        String studentName, course;

        void setData(String n, String c) {
            studentName = n;
            course = c;
        }

        void display() {
            System.out.println(collegeName + " " + studentName + " " + course);
        }
    }

    public static void main(String[] args) {
        College c = new College("ABC College");
        College.Admission a = c.new Admission();
        a.setData("John", "CS");
        a.display();
    }
}
```

12. Write a Java program that demonstrates method overloading to calculate the volume of different 3D shapes. Implement overloaded methods named calculateVolume() for a Cube (takes one side length), a RectangularCube (takes length, width, height), and a Sphere (takes radius).

Code: class Volume {

```
    double calculateVolume(double side) {
```

```
        return side * side * side;
    }

    double calculateVolume(double l, double w, double h) {
        return l * w * h;
    }

    double calculateVolumeSphere(double r) {
        return (4.0/3) * Math.PI * r*r*r;
    }

    public static void main(String[] args) {
        Volume v = new Volume();
        System.out.println(v.calculateVolume(3));
        System.out.println(v.calculateVolume(2,3,4));
        System.out.println(v.calculateVolumeSphere(3));
    }
}
```

Practical Set – 4

13. Design a base class Shape with two double data members d1 and d2 to store dimensions. Include a method getData(double d1, double d2) to initialize these dimensions. Create two derived classes,Triangle and Rectangle, which inherit from Shape. Each derived class should have its own method to calculate its specific area.

Code**: c**lass Shape {

```
   double d1, d2;


   void getData(double a, double b) {
      d1 = a; d2 = b;
   }
}


class Triangle extends Shape {
   double area() {
      return 0.5 * d1 * d2;
   }
}


class Rectangle extends Shape {
   double area() {
      return d1 * d2;
   }
}


class TestShape {
   public static void main(String[] args) {
      Triangle t = new Triangle();
```

```
        t.getData(10, 5);
        System.out.println(t.area());

        Rectangle r = new Rectangle();
        r.getData(4, 6);
        System.out.println(r.area());
    }
}
```

14. Define a base class BankAccount with common attributes like accountNumber,accountHolderName, and balance. Then, define two subclasses: SavingAccount and FixedDepositAccount, which inherit from BankAccount. Implement basic operations like openAccount(), deposit(), checkBalance(), and withdraw() in BankAccount. The SavingAccount should include a calculateInterest() method specific to savings accounts, and FixedDepositAccount should have a maturityAmount() method considering fixed deposit terms.

Code**:**

```
class BankAccount {
    String accNo, name;
    double balance;

    void deposit(double amt) { balance += amt; }
    void withdraw(double amt) { balance -= amt; }
    void checkBalance() { System.out.println(balance); }
}

class SavingAccount extends BankAccount {
    void calculateInterest() {
        System.out.println(balance * 0.04);
    }
}
```

```
class FixedDepositAccount extends BankAccount {
    void maturityAmount() {
        System.out.println(balance * 1.1);
    }
}
```

15. Create a base class named Employee that contains a method displayDetails() which prints general employee details such as name and department. Now create a subclass Manager that inherits from Employee and overrides the displayDetails() method to include additional information such as the manager's team size or project name. In the main method, create objects of both Employee and Manager classes and call the displayDetails() method using each object to show how Java determines which version of the method to execute at runtime.

Code**:**

```
class Employee {
    void displayDetails() {
        System.out.println("Employee");
    }
}

class Manager extends Employee {
    void displayDetails() {
        System.out.println("Manager with team");
    }
}

class Test {
    public static void main(String[] args) {
        Employee e = new Employee();
```

```
        Employee m = new Manager();

        e.displayDetails();
        m.displayDetails();
    }
}
```

Practical Set - 5

16. Given an interface Classify with a method String getDivision(double average). Implement this getDivision method in a class Result such that it returns "First Division" if the average is 60 or more.

Code: interface Classify {

```
    String getDivision(double avg);
}

class Result implements Classify {
    public String getDivision(double avg) {
        return avg >= 60 ? "First Division" : "Other";
    }
}
```

17. Write the Java code for an interface named Exam which declares a single abstract method Boolean isPassed(int mark). This method should take an integer mark as an argument. Write the Java code for another interface named Classify which declares a single abstract method String getDivision(double average). This method should take a double average as an argument. Create a class named Result that implements both the Exam and Classify interfaces. Provide concrete implementations for isPassed() and getDivision() methods. In your main method, create an instance of Result, set some marks and average, and demonstrate the use of both interface methods.

Code**:** interface Exam {

```
    boolean isPassed(int mark);
}

interface Classify {
    String getDivision(double avg);
}

class Result implements Exam, Classify {
    public boolean isPassed(int mark) {
        return mark >= 40;
    }

    public String getDivision(double avg) {
        return avg >= 60 ? "First" : "Second";
    }

    public static void main(String[] args) {
        Result r = new Result();
        System.out.println(r.isPassed(50));
        System.out.println(r.getDivision(65));
    }
}
```

18.Write a Java program to create an abstract class Vehicle with:

- **An abstract method fuelType() that returns the type of fuel used.**
- **An abstract method noOfWheels() that returns the number of wheels.**

Create two subclasses:

- **Car that uses Petrol/Diesel and has 4 wheels.**
- **Bike that uses Petrol and has 2 wheels.**

In the main method, create objects of Car and Bike, and display their fuel type and number of wheels.

Code**:** abstract class Vehicle {

```
    abstract String fuelType();
    abstract int noOfWheels();
}

class Car extends Vehicle {
    String fuelType() { return "Petrol"; }
    int noOfWheels() { return 4; }
}

class Bike extends Vehicle {
    String fuelType() { return "Petrol"; }
    int noOfWheels() { return 2; }
}

class TestVehicle {
    public static void main(String[] args) {
        Vehicle c = new Car();
        Vehicle b = new Bike();

        System.out.println(c.fuelType());
        System.out.println(b.noOfWheels());
    }
}
```

19. Write a Java program using packages to generate a mark sheet for students. Create a package student that contains a class Student with the following:

- **Data members: rollNo, name.**
- **A constructor to initialize student details.**
- **A method displayStudent() to display student information.**

Create another package exam that contains a class Result which:

- **Extends the Student class.**
- **Has data members: marks1, marks2, marks3.**
- **A method displayResult() that prints the student's mark sheet including total and average marks.**

In the main method (inside the exam package), create a student with marks and display the marksheet

Code:
```
class Student {
    int roll;
    String name;

    Student(int r, String n) {
        roll = r; name = n;
    }

    void displayStudent() {
        System.out.println(roll + " " + name);
    }
}

class Result extends Student {
    int m1, m2, m3;
```

```
    Result(int r, String n, int a, int b, int c) {
        super(r,n);
        m1=a; m2=b; m3=c;
    }

    void displayResult() {
        int total = m1+m2+m3;
        System.out.println(total + " " + total/3);
    }

    public static void main(String[] args) {
        Result r = new Result(1,"A",80,70,90);
        r.displayStudent();
        r.displayResult();
    }
}
```

Practical Set – 6

20. Take the value of denominator and numerator from user using command-line argument.Implement the concept of exception handling to manage all possible run-time error.

Code**:**

```
class Divide {
    public static void main(String[] args) {
        try {
            int a = Integer.parseInt(args[0]);
            int b = Integer.parseInt(args[1]);
            System.out.println(a / b);
        } catch (Exception e) {
            System.out.println("Error: " + e);
```

```
        }
    }
}
```

21. Write a Java program to create a class VotingApp where:

- **The method checkEligibility(int age) checks if a person is eligible to vote.**
- **If age < 18, explicitly throw the predefined exception IllegalArgumentException with the message "Age must be 18 or above to vote".**

In the main method, test the method with different age inputs.

- **Use a try-catch-finally block to handle exceptions.**
- **The finally block should always print "Validation process completed"**

Code**:**

```
class VotingApp {
    void checkEligibility(int age) {
        if (age < 18)
            throw new IllegalArgumentException("Age must be 18 or above to vote");
        else
            System.out.println("Eligible");
    }

    public static void main(String[] args) {
        VotingApp v = new VotingApp();
        try {
            v.checkEligibility(16);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        } finally {
            System.out.println("Validation process completed");
```

```
        }
    }
}
```

22. Define a custom exception class BookNotAvailableException that extends Exception.

- **Create a class Library with:**
- **An instance variable availableBooks (integer).**
- **A method issueBook(int count) that: o If count <= availableBooks, reduce the number of books and display "Book issued successfully".**

o Otherwise, throw BookNotAvailableException with the message "Requested books not available".

In the main() method:

- **Initialize the library with 3 available books.**
- **Try issuing 2 books (valid).**
- **Then try issuing 2 more books (should throw the custom exception).**

Code**:**

```
class BookNotAvailableException extends Exception {
    BookNotAvailableException(String msg) {
        super(msg);
    }
}


class Library {
    int books = 3;


    void issueBook(int count) throws BookNotAvailableException {
        if (count <= books) {
            books -= count;
            System.out.println("Issued");
```

```
        } else {
            throw new BookNotAvailableException("Not available");
        }
    }

    public static void main(String[] args) {
        try {
            Library l = new Library();
            l.issueBook(2);
            l.issueBook(2);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Practical Set – 7

23. Write a Java program that creates two threads:

- **First thread prints numbers from 1 to 10 at the interval of 1 second.**
- **Second thread prints numbers from 11 to 20 at the interval of 500 ms.**

Run both threads and display the output.

Code**:**

```
class T1 extends Thread {
    public void run() {
        for(int i=1;i<=10;i++){
            System.out.println(i);
            try { Thread.sleep(1000); } catch(Exception e){}
        }
```

```
    }
}

class T2 extends Thread {
    public void run() {
        for(int i=11;i<=20;i++){
            System.out.println(i);
            try { Thread.sleep(500); } catch(Exception e){}
        }
    }
}

class TestThread {
    public static void main(String[] args) {
        new T1().start();
        new T2().start();
    }
}
```

24. Write a Java program where Thread T1 prints 1 to 100, T2 prints 101 to 200 and T3 prints 201 to 300. Initiate execution of all the three threads but ensure that numbers should be printed sequentially

<u>Code</u>:

```
class T extends Thread {
    int start, end;

    T(int s, int e) { start=s; end=e; }

    public void run() {
        for(int i=start;i<=end;i++)
```

```
            System.out.println(i);
        }
    }

    class MainThread {
        public static void main(String[] args) throws Exception {
            T t1 = new T(1,100);
            T t2 = new T(101,200);
            T t3 = new T(201,300);

            t1.start(); t1.join();
            t2.start(); t2.join();
            t3.start();
        }
    }
```

25. Write a Java program where two threads print multiplication tables (e.g., Table of 5 and Table of 7). Use a synchronized method so that table outputs do not mix and remain consistent.

<u>Code</u>**:** class Table {

```
    synchronized void print(int n) {
        for(int i=1;i<=10;i++)
            System.out.println(n*i);
    }
}

class A extends Thread {
    Table t;
    A(Table t){this.t=t;}
```

```
    public void run(){t.print(5);}
}

class B extends Thread {
    Table t;
    B(Table t){this.t=t;}
    public void run(){t.print(7);}
}

class Test {
    public static void main(String[] args) {
        Table t = new Table();
        new A(t).start();
        new B(t).start();
    }
}
```

Practical Set – 8

26. Write a program that will count the number of characters, words, and lines in a file. Words are separated by whitespace characters. The file name should be passed as a command-line argument.

Code**:** import java.io.*;

```
class FileCount {
    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new FileReader(args[0]));
        int lines=0, words=0, chars=0;
        String line;
```

```
        while((line=br.readLine())!=null){
            lines++;
            words += line.split("\\s+").length;
            chars += line.length();
        }

        System.out.println(lines+" "+words+" "+chars);
    }
}
```

27. Create a file named students.txt. Write at least three student records (roll number, name, marks) into the file. Read the file content and display all student records on the console. Handle exceptions like IOException properly using try-catch-finally.

Code**:** import java.io.*;

```
class StudentFile {
    public static void main(String[] args) {
        try {
            BufferedWriter bw = new BufferedWriter(new
FileWriter("students.txt"));
            bw.write("1 A 80\n2 B 70\n3 C 90");
            bw.close();

            BufferedReader br = new BufferedReader(new
FileReader("students.txt"));
            String line;
            while((line=br.readLine())!=null)
                System.out.println(line);
```

```
        } catch(Exception e){
            System.out.println(e);
        }
    }
}
```

28. Write a Java program that reads a text file named data.txt. The program should count and display:The total number of lines, The total number of words, The total number of characters (excluding spaces and newline characters), Use FileReader / BufferedReader for reading the file. Handle exceptions like FileNotFoundException and IOException.

Code:

```
import java.io.*;

class FileStats {
    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new FileReader("data.txt"));
        int lines=0, words=0, chars=0;
        String line;

        while((line=br.readLine())!=null){
            lines++;
            words += line.split("\\s+").length;
            chars += line.replace(" ","").length();
        }

        System.out.println(lines+" "+words+" "+chars);
    }
}
```

Practical Set – 9

29. Write a Java program that uses an ArrayList<Integer> to store marks of students. Perform the following operations:

- **Add at least 5 marks.**
- **Display all marks.**
- **Find and display the highest and lowest marks using Collections.max() and Collections.min().**

Code: import java.util.*;

```
class Marks {
   public static void main(String[] args) {
      ArrayList<Integer> list = new ArrayList<>();
      list.add(50); list.add(60); list.add(70); list.add(80); list.add(90);


      System.out.println(list);
      System.out.println(Collections.max(list));
      System.out.println(Collections.min(list));
   }
}
```

30. Write a program that accepts a sentence from the user and counts the frequency of each word using a HashMap<String, Integer>. Display the results in the format: Input: "Java is fun and Java is powerful"

Output:

Java -> 2

is -> 2

fun -> 1

and -> 1

powerful –

Code: import java.util.*;

```
class WordFreq {
    public static void main(String[] args) {
        String str = "Java is fun and Java is powerful";
        HashMap<String,Integer> map = new HashMap<>();

        for(String w : str.split(" ")){
            map.put(w, map.getOrDefault(w,0)+1);
        }

        System.out.println(map);
    }
}
```

31. Write a Java program to simulate a Music Playlist using LinkedList<String>. Perform the following operations:

1. **Add songs to the playlist.**
2. **Display the full playlist.**
3. **Play the first song (remove from front).**
4. **Skip the last song (remove from end).**
5. **Display the updated playlist after each operation.**

Code**:** import java.util.*;

```
class Playlist {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.add("Song1");
        list.add("Song2");
        list.add("Song3");
```

```
        System.out.println(list);
        list.removeFirst();
        list.removeLast();
        System.out.println(list);
    }
}
```

32. Write a generic method searchElement that accepts a LinkedList<T> and an element T to search.Return true if the element exists, otherwise false.

- **Test with LinkedList<Integer> for roll numbers.**
- **Test with LinkedList<String> for names.**

Code:
```
import java.util.*;

class Search {
    static <T> boolean searchElement(LinkedList<T> list, T e) {
        return list.contains(e);
    }

    public static void main(String[] args) {
        LinkedList<Integer> l1 = new LinkedList<>();
        l1.add(1); l1.add(2);

        System.out.println(searchElement(l1,2));
    }
}
```

Practical Set – 10

33. Write a JavaFX program that displays five Text nodes vertically (stacked). For each Text:

● Use font Times New Roman, bold + italic, size 22 px.
● Assign a random color and random opacity (between 0.3 and 1.0) to each text.
● Center the texts horizontally in the window and add spacing between them. UI / Classes to use: VBox, Text, Font, Color, Random. On launch the window shows five vertically arranged lines (e.g., "Text 1", ... "Text 5") each with different color & opacity and same font style.

Code**:** import javafx.application.Application;

```
import javafx.scene.*;

import javafx.scene.layout.VBox;

import javafx.scene.text.*;

import javafx.scene.paint.Color;

import javafx.stage.Stage;

import java.util.Random;


public class TextFX extends Application {

   public void start(Stage stage) {

      VBox v = new VBox(10);

      Random r = new Random();


      for(int i=1;i<=5;i++){

         Text t = new Text("Text "+i);

         t.setFont(Font.font("Times New Roman", FontWeight.BOLD,
FontPosture.ITALIC,22));

         t.setFill(Color.color(r.nextDouble(), r.nextDouble(), r.nextDouble(),
0.3+r.nextDouble()*0.7));

         v.getChildren().add(t);

      }
```

```
        stage.setScene(new Scene(v,300,300));
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

34. Design a registration form UI with fields: Roll_NO (numeric), Name, Age (numeric), Email and a Submit button. Requirements:

- **Validate inputs on submit:**
  - **o RollNo and Age must be integers.**
  - **o Email must contain @ and . (basic check).**
- **On success show a confirmation Alert with entered data.**
- **On validation failure show an error alert describing the issue.**

UI / Classes to use: GridPane, TextField, Button, Alert, FileChooser, FileWriter/BufferedWriter.

Code:
```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;

public class RegistrationForm extends Application {

    @Override
    public void start(Stage stage) {
        GridPane grid = new GridPane();
        grid.setPadding(new Insets(20));
```

```
grid.setHgap(10);
grid.setVgap(10);

TextField rollField = new TextField();
TextField nameField = new TextField();
TextField ageField = new TextField();
TextField emailField = new TextField();

Button submitBtn = new Button("Submit");

grid.add(new Label("Roll No:"), 0, 0);
grid.add(rollField, 1, 0);

grid.add(new Label("Name:"), 0, 1);
grid.add(nameField, 1, 1);

grid.add(new Label("Age:"), 0, 2);
grid.add(ageField, 1, 2);

grid.add(new Label("Email:"), 0, 3);
grid.add(emailField, 1, 3);

grid.add(submitBtn, 1, 4);

submitBtn.setOnAction(e -> {
    try {
        int roll = Integer.parseInt(rollField.getText());
```

```
    int age = Integer.parseInt(ageField.getText());
    String name = nameField.getText();
    String email = emailField.getText();

    if (!email.contains("@") || !email.contains(".")) {
        throw new Exception("Invalid Email format");
    }

    Alert success = new Alert(Alert.AlertType.INFORMATION);
    success.setTitle("Success");
    success.setHeaderText("Registration Successful");
    success.setContentText(
            "Roll: " + roll +
            "\nName: " + name +
            "\nAge: " + age +
            "\nEmail: " + email
    );
    success.show();

} catch (NumberFormatException ex) {
    Alert error = new Alert(Alert.AlertType.ERROR);
    error.setContentText("Roll No and Age must be integers");
    error.show();
} catch (Exception ex) {
    Alert error = new Alert(Alert.AlertType.ERROR);
    error.setContentText(ex.getMessage());
    error.show();
```

```
        }
    });

    stage.setTitle("Registration Form");
    stage.setScene(new Scene(grid, 350, 250));
    stage.show();
  }

  public static void main(String[] args) {
    launch(args);
  }
}
```

35. Write a JavaFX program that displays a bar chart to represent the percentage distribution of overall grades using Rectangle shapes.

- **Projects: 20%, displayed in Red**
- **Quizzes: 10%, displayed in Blue**
- **Midterm Exams: 30%, displayed in Green**
- **Final Exam: 40%, displayed in Orange**

Requirements

1. Each category should be displayed with a labeled bar.
2. Bars should be proportional in height to the percentage.
3. Use the Rectangle class to create the bars.
4. Display the labels (e.g., "Projects — 20%") under each bar.
5. Arrange the bars horizontally in the scene using an HBox or Pane.

Code**:**

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.control.Label;
```

```
import javafx.geometry.Pos;
import javafx.stage.Stage;

public class BarChartFX extends Application {

    @Override
    public void start(Stage stage) {
        HBox root = new HBox(20);
        root.setAlignment(Pos.BOTTOM_CENTER);

        root.getChildren().add(createBar("Projects — 20%", 20, Color.RED));
        root.getChildren().add(createBar("Quizzes — 10%", 10, Color.BLUE));
        root.getChildren().add(createBar("Midterm — 30%", 30, Color.GREEN));
        root.getChildren().add(createBar("Final — 40%", 40, Color.ORANGE));

        Scene scene = new Scene(root, 500, 300);
        stage.setTitle("Grade Distribution");
        stage.setScene(scene);
        stage.show();
    }

    private VBox createBar(String labelText, int percentage, Color color) {
        int maxHeight = 200; // scaling factor
        Rectangle bar = new Rectangle(50, percentage * (maxHeight / 40.0));
        bar.setFill(color);

        Label label = new Label(labelText);
```

```
        VBox vbox = new VBox(5);
        vbox.setAlignment(Pos.BOTTOM_CENTER);
        vbox.getChildren().addAll(bar, label);

        return vbox;
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```